In [1]:
```
!python --version
```

```
Python 3.12.7
```

In [2]:
```
# !pip install matplotlib
# !pip install seaborn
# !pip install scipy
# !pip install scikit-learn
```

In [3]:
```
# Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from itertools import combinations
from sklearn.linear_model import LinearRegression
from scipy import stats
import warnings

# Alias for stats to avoid confusion later
from scipy import stats as scipy_stats

# Suppress warnings to keep output clean
warnings.filterwarnings('ignore')

# ---------------------------
# Load the dataset
# ---------------------------
df = pd.read_csv('FE-GWP1_model_selection_1.csv')

print("Dataset loaded successfully!")
print(f"Shape of the dataset: {df.shape}")
print("\nHere are the first few rows:")
print(df.head())

# ---------------------------
# Define predictors and response
# ---------------------------
X = df[['X1', 'X2', 'X3', 'X4', 'X5']]  # predictor variables
y = df['Y']                              # dependent variable

# Number of observations
n = len(y)

# List of predictor names for convenience
predictors = ['X1', 'X2', 'X3', 'X4', 'X5']
```

```
Dataset loaded successfully!
Shape of the dataset: (100, 6)

Here are the first few rows:
          Y        X1        X2        X3        X4        X5
0  3.388410  0.017954 -0.800583 -0.352454  2.187210  1.014887
1  0.287191  0.083057 -0.597947 -0.357639 -1.630284  0.221841
2  3.989645 -0.923437 -1.386575  1.180202  0.632606 -1.576638
3 -2.959602 -0.313775  2.955133 -1.798692 -2.117621  0.159291
4  0.529773  0.388996  1.019611  0.472062  0.590497  0.877048
```

In [4]:
```
np.random.seed(77777)
```

In [5]:
```
arparams1 = np.r_[1, -np.array([0.6, 0.3])]
maparams1 = np.r_[1, np.array([0.0, 0.0])]
```

In [6]:
```
arparams1
```

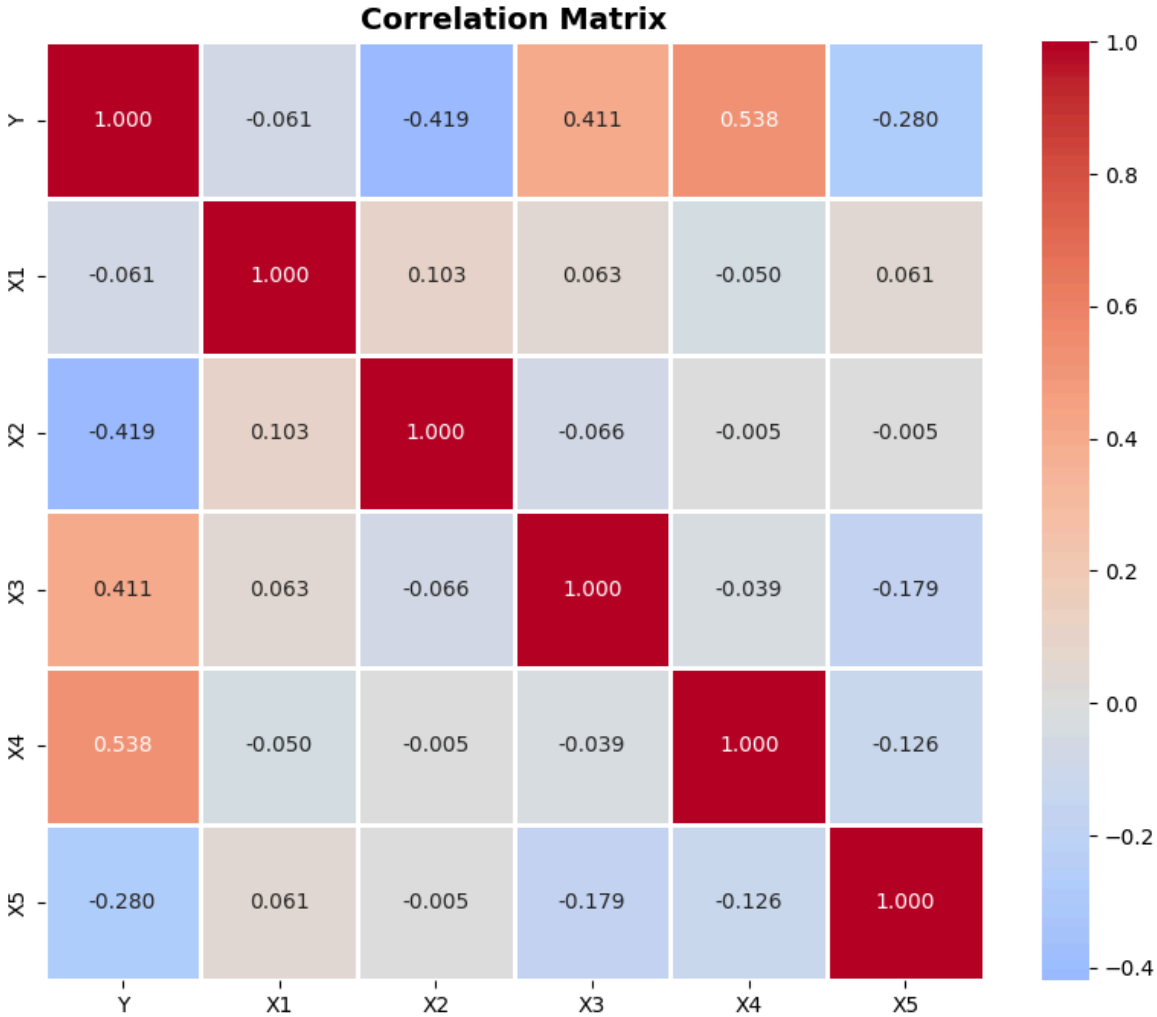
Out[6]:
```
array([ 1. , -0.6, -0.3])
```

In [7]:
```
maparams1
```

Out[7]:
```
array([1., 0., 0.])
```

In [8]:
```
# ---------------------------
# Descriptive statistics
# ---------------------------
print("Some basic descriptive statistics for the dataset:")
print(df.describe())

# ---------------------------
# Correlation analysis
# ---------------------------
corr_matrix = df.corr()

print("\nCorrelation matrix for all variables:")
print(corr_matrix)

print("\nCorrelations of each predictor with Y (sorted):")
print(corr_matrix['Y'].sort_values(ascending=False))

# ---------------------------
# Visualize correlation matrix
# ---------------------------
plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', center=0,
            square=True, linewidths=1, fmt='.3f')
plt.title('Correlation Matrix', fontsize=14, fontweight='bold')
plt.show()
```

```
Some basic descriptive statistics for the dataset:
                Y          X1          X2          X3          X4          X5
count  100.000000  100.000000  100.000000  100.000000  100.000000  100.000000
mean     1.257388    0.026830    0.084613   -0.016037    0.122374   -0.201661
std      1.436655    0.481708    0.962145    0.976559    1.076935    1.073358
min     -2.959602   -1.275230   -2.041959   -2.228483   -2.697316   -3.526357
25%      0.349470   -0.316585   -0.676951   -0.572893   -0.431023   -0.960214
50%      1.278526    0.040237    0.040447    0.056879    0.279594   -0.185795
75%      2.152954    0.373899    0.748166    0.636408    0.747868    0.506592
max      5.176803    1.083062    2.955133    1.816825    3.167298    2.499820

Correlation matrix for all variables:
           Y        X1        X2        X3        X4        X5
Y   1.000000 -0.061400 -0.419326  0.411351  0.538308 -0.279807
X1 -0.061400  1.000000  0.103063  0.062876 -0.050210  0.061230
X2 -0.419326  0.103063  1.000000 -0.065582 -0.004993 -0.005223
X3  0.411351  0.062876 -0.065582  1.000000 -0.038886 -0.178590
X4  0.538308 -0.050210 -0.004993 -0.038886  1.000000 -0.125909
X5 -0.279807  0.061230 -0.005223 -0.178590 -0.125909  1.000000

Correlations of each predictor with Y (sorted):
Y     1.000000
X4    0.538308
X3    0.411351
X1   -0.061400
X5   -0.279807
X2   -0.419326
Name: Y, dtype: float64
```



In [9]:
```
def calculate_model_stats(X_subset, y, predictor_names):
    """
    Fit a linear regression model on the given predictors and return
    key statistics useful for model comparison.
    """

    n = len(y)                  # number of observations
    k = len(predictor_names)    # number of predictors in this model

    # Fit the linear regression model
    model = LinearRegression()
    model.fit(X_subset, y)

    # Get model predictions and residuals
    y_pred = model.predict(X_subset)
    residuals = y - y_pred

    # Compute total and residual sum of squares
    ss_total = np.sum((y - np.mean(y)) ** 2)
    ss_residual = np.sum(residuals ** 2)

    # R-squared
    r_squared = 1 - ss_residual / ss_total

    # Adjusted R-squared and mean squared error (MSE)
    # The check below is just for safety
    if n - k - 1 <= 0:
        adj_r_squared = np.nan
        mse = np.nan
    else:
        adj_r_squared = 1 - (1 - r_squared) * (n - 1) / (n - k - 1)
        mse = ss_residual / (n - k - 1)

    # Akaike and Bayesian information criteria
    aic = n * np.log(ss_residual / n) + 2 * (k + 1)
    bic = n * np.log(ss_residual / n) + (k + 1) * np.log(n)

    # Return all the important stats as a dictionary
    return {
        "predictors": predictor_names,
        "k": k,
        "r_squared": r_squared,
        "adj_r_squared": adj_r_squared,
        "aic": aic,
        "bic": bic,
        "mse": mse,
        "ss_residual": ss_residual,
        "coefficients": model.coef_,
        "intercept": model.intercept_,
        "y_pred": y_pred,
        "residuals": residuals
    }
```

In [10]:
```
print("\n" + "=" * 80)
print("EXHAUSTIVE SEARCH - CHECKING ALL POSSIBLE MODELS")
print("=" * 80)

# We'll store stats for every model we test
all_models = []
```

```
# Try all combinations of predictors, from 1 up to all of them
for i in range(1, len(predictors) + 1):
    for combo in combinations(predictors, i):
        cols = list(combo)
        X_subset = X[cols]

        # Get stats for this combination
        model_stats = calculate_model_stats(X_subset, y, cols)
        all_models.append(model_stats)

# Grab stats for the full model (needed for Mallows' Cp)
stats_full = [m for m in all_models if m["k"] == len(predictors)][0]
full_model_mse = stats_full["mse"]

# Calculate Mallows' Cp for all models
for m in all_models:
    k = m["k"]
    # Cp = SSE_model / MSE_full - (n - 2*(p+1))
    cp = m["ss_residual"] / full_model_mse - (n - 2 * (k + 1))
    m["mallows_cp"] = cp

# Build a summary table for easier viewing
summary_rows = []
for m in all_models:
    summary_rows.append({
        "Model": " + ".join(m["predictors"]),
        "k": m["k"],
        "R²": m["r_squared"],
        "Adj R²": m["adj_r_squared"],
        "AIC": m["aic"],
        "BIC": m["bic"],
        "Mallows Cp": m["mallows_cp"],
        "MSE": m["mse"]
    })

summary_df = pd.DataFrame(summary_rows)

print(f"\nTotal number of models evaluated: {len(all_models)}")

# Display top models by various criteria
print("\nTop 5 models by Adjusted R²:")
print(summary_df.nlargest(5, "Adj R²").to_string(index=False))

print("\nTop 5 models by AIC (lower is better):")
print(summary_df.nsmallest(5, "AIC").to_string(index=False))

print("\nTop 5 models by BIC (lower is better):")
print(summary_df.nsmallest(5, "BIC").to_string(index=False))

print("\nTop 5 models by Mallows' Cp (closer to k+1 is better):")
summary_df["Cp_distance"] = np.abs(summary_df["Mallows Cp"] - (summary_df["k"] +
print(
    summary_df
    .nsmallest(5, "Cp_distance")[["Model", "k", "R²", "Adj R²", "AIC", "BIC", "M
    .to_string(index=False)
)

# Save the summary to a CSV so we can check it later
summary_df.to_csv("all_models_summary.csv", index=False)
```

```
================================================================================
EXHAUSTIVE SEARCH - CHECKING ALL POSSIBLE MODELS
================================================================================

Total number of models evaluated: 31

Top 5 models by Adjusted R²:
                Model  k       R²   Adj R²        AIC        BIC  Mallows Cp
MSE
     X2 + X3 + X4 + X5  4 0.648763 0.633974 -23.171022 -10.145172    4.022709 0.7
55469
X1 + X2 + X3 + X4 + X5  5 0.648848 0.630170 -21.195178  -5.564157    6.000000 0.7
63321
          X2 + X3 + X4  3 0.628256 0.616639 -19.496653  -9.075972    7.512179 0.7
91247
     X1 + X2 + X3 + X4  4 0.628628 0.612991 -17.596610  -4.570759    9.412759 0.7
98778
          X2 + X4 + X5  3 0.510106 0.494796   8.101890  18.522571   39.140001 1.0
42729

Top 5 models by AIC (lower is better):
                Model  k       R²   Adj R²        AIC        BIC  Mallows Cp
MSE
     X2 + X3 + X4 + X5  4 0.648763 0.633974 -23.171022 -10.145172    4.022709 0.7
55469
X1 + X2 + X3 + X4 + X5  5 0.648848 0.630170 -21.195178  -5.564157    6.000000 0.7
63321
          X2 + X3 + X4  3 0.628256 0.616639 -19.496653  -9.075972    7.512179 0.7
91247
     X1 + X2 + X3 + X4  4 0.628628 0.612991 -17.596610  -4.570759    9.412759 0.7
98778
          X2 + X4 + X5  3 0.510106 0.494796   8.101890  18.522571   39.140001 1.0
42729

Top 5 models by BIC (lower is better):
                Model  k       R²   Adj R²        AIC        BIC  Mallows Cp
MSE
     X2 + X3 + X4 + X5  4 0.648763 0.633974 -23.171022 -10.145172    4.022709 0.7
55469
          X2 + X3 + X4  3 0.628256 0.616639 -19.496653  -9.075972    7.512179 0.7
91247
X1 + X2 + X3 + X4 + X5  5 0.648848 0.630170 -21.195178  -5.564157    6.000000 0.7
63321
     X1 + X2 + X3 + X4  4 0.628628 0.612991 -17.596610  -4.570759    9.412759 0.7
98778
          X2 + X4 + X5  3 0.510106 0.494796   8.101890  18.522571   39.140001 1.0
42729

Top 5 models by Mallows' Cp (closer to k+1 is better):
                Model  k       R²   Adj R²        AIC        BIC  Mallows Cp
X1 + X2 + X3 + X4 + X5  5 0.648848 0.630170 -21.195178  -5.564157    6.000000
     X2 + X3 + X4 + X5  4 0.648763 0.633974 -23.171022 -10.145172    4.022709
          X2 + X3 + X4  3 0.628256 0.616639 -19.496653  -9.075972    7.512179
     X1 + X2 + X3 + X4  4 0.628628 0.612991 -17.596610  -4.570759    9.412759
          X2 + X4 + X5  3 0.510106 0.494796   8.101890  18.522571   39.140001
```

In [11]:
```
def forward_selection(X, y, predictors, criterion='adj_r_squared'):
    """
    Forward stepwise selection.
    Starts with no predictors and adds one at a time,
```

```
    choosing the variable that best improves the model.
    """
    remaining = list(predictors)
    selected = []
    history = []

    print(f"\nStarting Forward Selection using {criterion.replace('_', ' ').uppe
    print("-" * 80)

    while remaining:
        best_score = -np.inf if criterion == 'adj_r_squared' else np.inf
        best_var = None
        best_stats = None

        for var in remaining:
            trial_vars = selected + [var]
            X_subset = X[trial_vars]
            stats = calculate_model_stats(X_subset, y, trial_vars)

            # Compute Mallows' Cp for this subset
            k = stats['k']
            stats['mallows_cp'] = stats['ss_residual'] / full_model_mse - (n - 2

            score = stats[criterion]

            if (criterion == 'adj_r_squared' and score > best_score) or \
               (criterion in ['aic', 'bic'] and score < best_score):
                best_score = score
                best_var = var
                best_stats = stats

        # Stop if adding a variable doesn't improve Adjusted R²
        if selected and criterion == 'adj_r_squared' and best_score <= history[-
            print(f"\nStopping: Adding {best_var} does not improve Adjusted R²."
            break

        if best_var is None:
            break

        selected.append(best_var)
        remaining.remove(best_var)
        history.append(best_stats)

        print(f"\nStep {len(selected)}: Added {best_var}")
        print(f"  Current model: {' + '.join(selected)}")
        print(f"  R² = {best_stats['r_squared']:.6f}, Adj R² = {best_stats['adj_
        print(f"  AIC = {best_stats['aic']:.4f}, BIC = {best_stats['bic']:.4f}")
        print(f"  Mallows' Cp = {best_stats['mallows_cp']:.4f}")

    return selected, history


def backward_elimination(X, y, predictors, criterion='adj_r_squared'):
    """
    Backward stepwise elimination.
    Starts with all predictors and removes the least useful one at a time.
    """
    selected = list(predictors)
    history = []
```

```
    print(f"\nStarting Backward Elimination using {criterion.replace('_', ' ').u
    print("-" * 80)

    # Initial full model stats
    stats_full_model = calculate_model_stats(X[selected], y, selected)
    k = stats_full_model['k']
    stats_full_model['mallows_cp'] = stats_full_model['ss_residual'] / full_mode
    history.append(stats_full_model)

    print(f"\nInitial model: {' + '.join(selected)}")
    print(f"  R² = {stats_full_model['r_squared']:.6f}, Adj R² = {stats_full_mod
    print(f"  AIC = {stats_full_model['aic']:.4f}, BIC = {stats_full_model['bic'
    print(f"  Mallows' Cp = {stats_full_model['mallows_cp']:.4f}")

    while len(selected) > 1:
        worst_score = np.inf if criterion in ['aic', 'bic'] else -np.inf
        worst_var = None
        best_stats_after_removal = None

        for var in selected:
            trial_vars = [p for p in selected if p != var]
            stats = calculate_model_stats(X[trial_vars], y, trial_vars)
            k = stats['k']
            stats['mallows_cp'] = stats['ss_residual'] / full_model_mse - (n - 2
            score = stats[criterion]

            if (criterion == 'adj_r_squared' and score > worst_score) or \
               (criterion in ['aic', 'bic'] and score < worst_score):
                worst_score = score
                worst_var = var
                best_stats_after_removal = stats

        # Check if removal worsens the model
        if criterion == 'adj_r_squared' and best_stats_after_removal['adj_r_squa
            print(f"\nStopping: Removing {worst_var} decreases Adjusted R².")
            break
        if criterion == 'aic' and best_stats_after_removal['aic'] > history[-1][
            print(f"\nStopping: Removing {worst_var} increases AIC.")
            break
        if criterion == 'bic' and best_stats_after_removal['bic'] > history[-1][
            print(f"\nStopping: Removing {worst_var} increases BIC.")
            break

        selected.remove(worst_var)
        history.append(best_stats_after_removal)

        print(f"\nStep {len(history)}: Removed {worst_var}")
        print(f"  Current model: {' + '.join(selected)}")
        print(f"  R² = {best_stats_after_removal['r_squared']:.6f}, Adj R² = {be
        print(f"  AIC = {best_stats_after_removal['aic']:.4f}, BIC = {best_stats
        print(f"  Mallows' Cp = {best_stats_after_removal['mallows_cp']:.4f}")

    return selected, history
```

In [12]:

```
print("\n" + "="*80)
print("STEPWISE SELECTION EXECUTION")
print("="*80)

# ---------- Forward Selection ----------
print("\nRunning forward selection using different criteria...\n")
```

```python
# Forward selection using Adjusted R²
selected_fwd_adj_r2, history_fwd_adj_r2 = forward_selection(
    X, y, predictors, 'adj_r_squared'
)

# Forward selection using AIC
selected_fwd_aic, history_fwd_aic = forward_selection(
    X, y, predictors, 'aic'
)

# Forward selection using BIC
selected_fwd_bic, history_fwd_bic = forward_selection(
    X, y, predictors, 'bic'
)

print("\n" + "="*80)
print("BACKWARD ELIMINATION EXECUTION")
print("="*80)

# ---------- Backward Elimination ----------
print("\nRunning backward elimination using different criteria...\n")

# Backward elimination using Adjusted R²
selected_back_adj_r2, history_back_adj_r2 = backward_elimination(
    X, y, predictors, 'adj_r_squared'
)

# Backward elimination using AIC
selected_back_aic, history_back_aic = backward_elimination(
    X, y, predictors, 'aic'
)

# Backward elimination using BIC
selected_back_bic, history_back_bic = backward_elimination(
    X, y, predictors, 'bic'
)
```

```
================================================================================
STEPWISE SELECTION EXECUTION
================================================================================

Running forward selection using different criteria...


Starting Forward Selection using ADJ R SQUARED
--------------------------------------------------------------------------------

Step 1: Added X4
  Current model: X4
  R² = 0.289776, Adj R² = 0.282529
  AIC = 41.2410, BIC = 46.4513
  Mallows' Cp = 94.1202

Step 2: Added X3
  Current model: X4 + X3
  R² = 0.476928, Adj R² = 0.466143
  AIC = 12.6548, BIC = 20.4703
  Mallows' Cp = 46.0213

Step 3: Added X2
  Current model: X4 + X3 + X2
  R² = 0.628256, Adj R² = 0.616639
  AIC = -19.4967, BIC = -9.0760
  Mallows' Cp = 7.5122

Step 4: Added X5
  Current model: X4 + X3 + X2 + X5
  R² = 0.648763, Adj R² = 0.633974
  AIC = -23.1710, BIC = -10.1452
  Mallows' Cp = 4.0227

Stopping: Adding X1 does not improve Adjusted R².

Starting Forward Selection using AIC
--------------------------------------------------------------------------------

Step 1: Added X4
  Current model: X4
  R² = 0.289776, Adj R² = 0.282529
  AIC = 41.2410, BIC = 46.4513
  Mallows' Cp = 94.1202

Step 2: Added X3
  Current model: X4 + X3
  R² = 0.476928, Adj R² = 0.466143
  AIC = 12.6548, BIC = 20.4703
  Mallows' Cp = 46.0213

Step 3: Added X2
  Current model: X4 + X3 + X2
  R² = 0.628256, Adj R² = 0.616639
  AIC = -19.4967, BIC = -9.0760
  Mallows' Cp = 7.5122

Step 4: Added X5
  Current model: X4 + X3 + X2 + X5
  R² = 0.648763, Adj R² = 0.633974
```

```
  AIC = -23.1710, BIC = -10.1452
  Mallows' Cp = 4.0227

Step 5: Added X1
  Current model: X4 + X3 + X2 + X5 + X1
  R² = 0.648848, Adj R² = 0.630170
  AIC = -21.1952, BIC = -5.5642
  Mallows' Cp = 6.0000

Starting Forward Selection using BIC
--------------------------------------------------------------------------------

Step 1: Added X4
  Current model: X4
  R² = 0.289776, Adj R² = 0.282529
  AIC = 41.2410, BIC = 46.4513
  Mallows' Cp = 94.1202

Step 2: Added X3
  Current model: X4 + X3
  R² = 0.476928, Adj R² = 0.466143
  AIC = 12.6548, BIC = 20.4703
  Mallows' Cp = 46.0213

Step 3: Added X2
  Current model: X4 + X3 + X2
  R² = 0.628256, Adj R² = 0.616639
  AIC = -19.4967, BIC = -9.0760
  Mallows' Cp = 7.5122

Step 4: Added X5
  Current model: X4 + X3 + X2 + X5
  R² = 0.648763, Adj R² = 0.633974
  AIC = -23.1710, BIC = -10.1452
  Mallows' Cp = 4.0227

Step 5: Added X1
  Current model: X4 + X3 + X2 + X5 + X1
  R² = 0.648848, Adj R² = 0.630170
  AIC = -21.1952, BIC = -5.5642
  Mallows' Cp = 6.0000

================================================================================
BACKWARD ELIMINATION EXECUTION
================================================================================

Running backward elimination using different criteria...


Starting Backward Elimination using ADJ R SQUARED
--------------------------------------------------------------------------------

Initial model: X1 + X2 + X3 + X4 + X5
  R² = 0.648848, Adj R² = 0.630170
  AIC = -21.1952, BIC = -5.5642
  Mallows' Cp = 6.0000

Step 2: Removed X1
  Current model: X2 + X3 + X4 + X5
  R² = 0.648763, Adj R² = 0.633974
```

```
  AIC = -23.1710, BIC = -10.1452
  Mallows' Cp = 4.0227

Stopping: Removing X5 decreases Adjusted R².

Starting Backward Elimination using AIC
--------------------------------------------------------------------------------

Initial model: X1 + X2 + X3 + X4 + X5
  R² = 0.648848, Adj R² = 0.630170
  AIC = -21.1952, BIC = -5.5642
  Mallows' Cp = 6.0000

Step 2: Removed X1
  Current model: X2 + X3 + X4 + X5
  R² = 0.648763, Adj R² = 0.633974
  AIC = -23.1710, BIC = -10.1452
  Mallows' Cp = 4.0227

Stopping: Removing X5 increases AIC.

Starting Backward Elimination using BIC
--------------------------------------------------------------------------------

Initial model: X1 + X2 + X3 + X4 + X5
  R² = 0.648848, Adj R² = 0.630170
  AIC = -21.1952, BIC = -5.5642
  Mallows' Cp = 6.0000

Step 2: Removed X1
  Current model: X2 + X3 + X4 + X5
  R² = 0.648763, Adj R² = 0.633974
  AIC = -23.1710, BIC = -10.1452
  Mallows' Cp = 4.0227

Stopping: Removing X5 increases BIC.
```

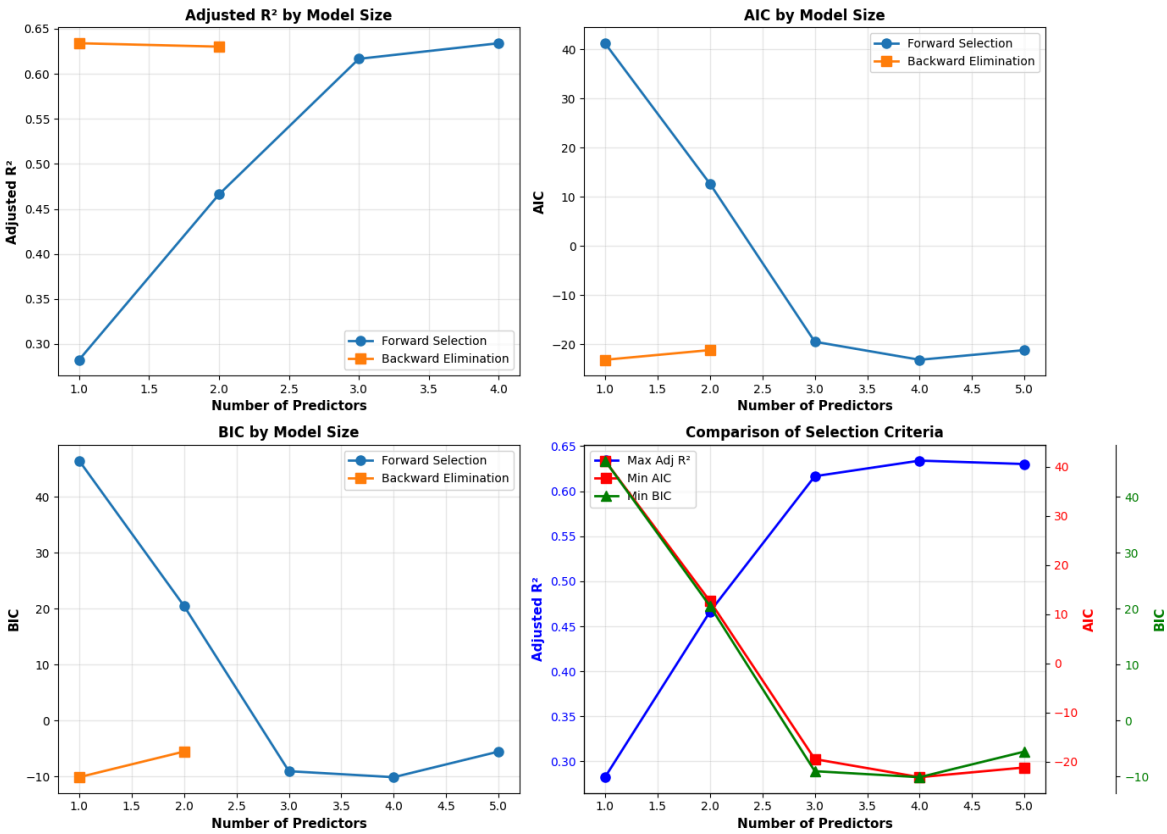
In [13]:

```python
print("\n" + "="*80)
print("SUMMARY OF MODEL SELECTION RESULTS")
print("="*80)

print("\n1. FORWARD SELECTION RESULTS:")
print(f"   - Best model by Adj R²: {' + '.join(selected_fwd_adj_r2)}")
print(f"   - Best model by AIC: {' + '.join(selected_fwd_aic)}")
print(f"   - Best model by BIC: {' + '.join(selected_fwd_bic)}")

print("\n2. BACKWARD ELIMINATION RESULTS:")
print(f"   - Best model by Adj R²: {' + '.join(selected_back_adj_r2)}")
print(f"   - Best model by AIC: {' + '.join(selected_back_aic)}")
print(f"   - Best model by BIC: {' + '.join(selected_back_bic)}")

print("\n3. EXHAUSTIVE SEARCH RESULTS:")
best_adj_r2_model = summary_df.loc[summary_df['Adj R²'].idxmax()]
best_aic_model = summary_df.loc[summary_df['AIC'].idxmin()]
best_bic_model = summary_df.loc[summary_df['BIC'].idxmin()]
best_cp_model = summary_df.loc[summary_df['Cp_distance'].idxmin()]

print(f"   - Best model by Adj R²: {best_adj_r2_model['Model']}")
print(f"   - Best model by AIC: {best_aic_model['Model']}")
print(f"   - Best model by BIC: {best_bic_model['Model']}")
```

```
print(f"   - Best model by Mallows' Cp: {best_cp_model['Model']}")

# Create visualization of model selection paths
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Plot 1: Adj R² progression
ax1 = axes[0, 0]
forward_adj_r2_values = [h['adj_r_squared'] for h in history_fwd_adj_r2]
backward_adj_r2_values = [h['adj_r_squared'] for h in history_back_adj_r2]
ax1.plot(range(1, len(forward_adj_r2_values) + 1), forward_adj_r2_values,
         'o-', label='Forward Selection', linewidth=2, markersize=8)
ax1.plot(range(len(backward_adj_r2_values), 0, -1), backward_adj_r2_values,
         's-', label='Backward Elimination', linewidth=2, markersize=8)
ax1.set_xlabel('Number of Predictors', fontsize=11, fontweight='bold')
ax1.set_ylabel('Adjusted R²', fontsize=11, fontweight='bold')
ax1.set_title('Adjusted R² by Model Size', fontsize=12, fontweight='bold')
ax1.legend()
ax1.grid(True, alpha=0.3)

# Plot 2: AIC progression
ax2 = axes[0, 1]
forward_aic_values = [h['aic'] for h in history_fwd_aic]
backward_aic_values = [h['aic'] for h in history_back_aic]
ax2.plot(range(1, len(forward_aic_values) + 1), forward_aic_values,
         'o-', label='Forward Selection', linewidth=2, markersize=8)
ax2.plot(range(len(backward_aic_values), 0, -1), backward_aic_values,
         's-', label='Backward Elimination', linewidth=2, markersize=8)
ax2.set_xlabel('Number of Predictors', fontsize=11, fontweight='bold')
ax2.set_ylabel('AIC', fontsize=11, fontweight='bold')
ax2.set_title('AIC by Model Size', fontsize=12, fontweight='bold')
ax2.legend()
ax2.grid(True, alpha=0.3)

# Plot 3: BIC progression
ax3 = axes[1, 0]
forward_bic_values = [h['bic'] for h in history_fwd_bic]
backward_bic_values = [h['bic'] for h in history_back_bic]
ax3.plot(range(1, len(forward_bic_values) + 1), forward_bic_values,
         'o-', label='Forward Selection', linewidth=2, markersize=8)
ax3.plot(range(len(backward_bic_values), 0, -1), backward_bic_values,
         's-', label='Backward Elimination', linewidth=2, markersize=8)
ax3.set_xlabel('Number of Predictors', fontsize=11, fontweight='bold')
ax3.set_ylabel('BIC', fontsize=11, fontweight='bold')
ax3.set_title('BIC by Model Size', fontsize=12, fontweight='bold')
ax3.legend()
ax3.grid(True, alpha=0.3)

# Plot 4: Comparison of criteria for all models
ax4 = axes[1, 1]
models_by_size = summary_df.groupby('k').agg({
    'Adj R²': 'max',
    'AIC': 'min',
    'BIC': 'min'
}).reset_index()

ax4_twin1 = ax4.twinx()
ax4_twin2 = ax4.twinx()
ax4_twin2.spines['right'].set_position(('outward', 60))

p1 = ax4.plot(models_by_size['k'], models_by_size['Adj R²'],
```

```
                'o-', color='blue', label='Max Adj R²', linewidth=2, markersize=8)
p2 = ax4_twin1.plot(models_by_size['k'], models_by_size['AIC'],
                    's-', color='red', label='Min AIC', linewidth=2, markersize=
p3 = ax4_twin2.plot(models_by_size['k'], models_by_size['BIC'],
                    '^-', color='green', label='Min BIC', linewidth=2, markersiz

ax4.set_xlabel('Number of Predictors', fontsize=11, fontweight='bold')
ax4.set_ylabel('Adjusted R²', color='blue', fontsize=11, fontweight='bold')
ax4_twin1.set_ylabel('AIC', color='red', fontsize=11, fontweight='bold')
ax4_twin2.set_ylabel('BIC', color='green', fontsize=11, fontweight='bold')
ax4.set_title('Comparison of Selection Criteria', fontsize=12, fontweight='bold'

ax4.tick_params(axis='y', labelcolor='blue')
ax4_twin1.tick_params(axis='y', labelcolor='red')
ax4_twin2.tick_params(axis='y', labelcolor='green')

lines = p1 + p2 + p3
labels = [l.get_label() for l in lines]
ax4.legend(lines, labels, loc='best')
ax4.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

```
================================================================================
SUMMARY OF MODEL SELECTION RESULTS
================================================================================

1. FORWARD SELECTION RESULTS:
   - Best model by Adj R²: X4 + X3 + X2 + X5
   - Best model by AIC: X4 + X3 + X2 + X5 + X1
   - Best model by BIC: X4 + X3 + X2 + X5 + X1

2. BACKWARD ELIMINATION RESULTS:
   - Best model by Adj R²: X2 + X3 + X4 + X5
   - Best model by AIC: X2 + X3 + X4 + X5
   - Best model by BIC: X2 + X3 + X4 + X5

3. EXHAUSTIVE SEARCH RESULTS:
   - Best model by Adj R²: X2 + X3 + X4 + X5
   - Best model by AIC: X2 + X3 + X4 + X5
   - Best model by BIC: X2 + X3 + X4 + X5
   - Best model by Mallows' Cp: X1 + X2 + X3 + X4 + X5
```



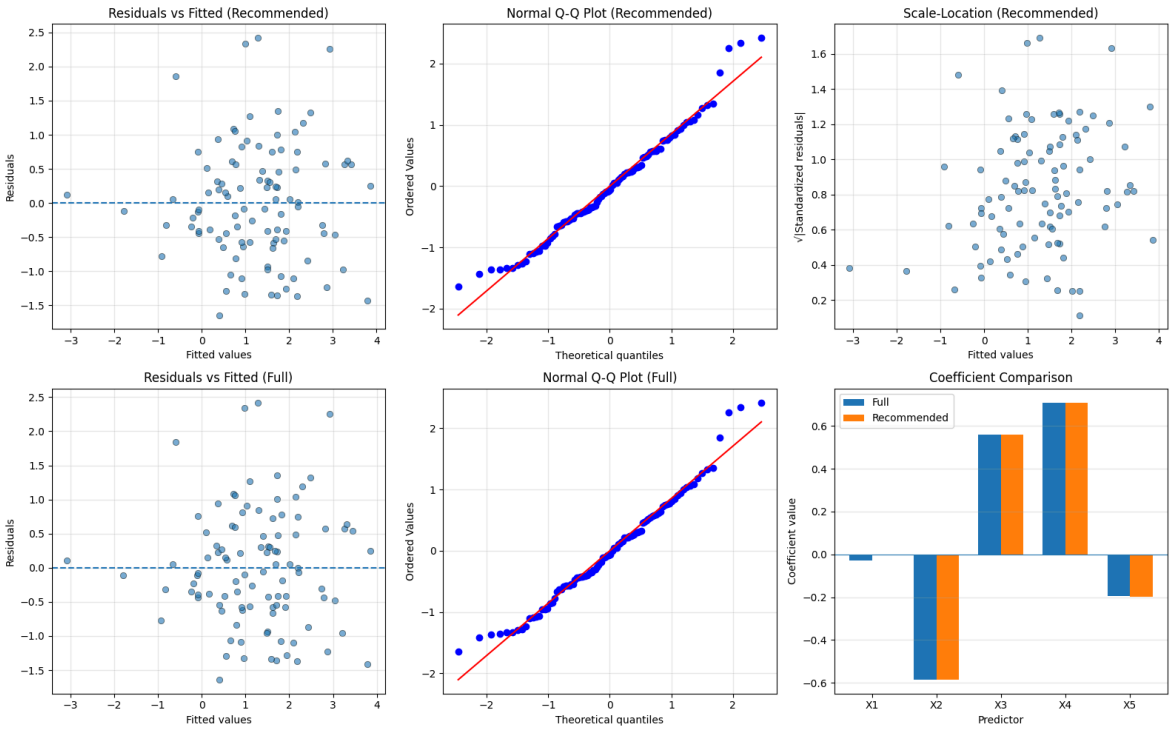
In [14]:

```
print("\n" + "=" * 80)
print("FINAL MODEL DIAGNOSTICS AND COMPARISON")
print("=" * 80)

# recommended model vs full model
rec_vars = ["X2", "X3", "X4", "X5"]
full_vars = ["X1", "X2", "X3", "X4", "X5"]

stats_rec = calculate_model_stats(X[rec_vars], y, rec_vars)
stats_full = calculate_model_stats(X[full_vars], y, full_vars)

# keep fitted values and residuals handy
y_hat_rec = stats_rec["y_pred"]
resid_rec = stats_rec["residuals"]

y_hat_full = stats_full["y_pred"]
resid_full = stats_full["residuals"]

# ---------------------------------------------------
# Recommended model output
# ---------------------------------------------------
print("\nRECOMMENDED MODEL: Y ~ X2 + X3 + X4 + X5")
print("-" * 80)
print(f"Intercept: {stats_rec['intercept']:.6f}")
print("Coefficients:")
for i, v in enumerate(rec_vars):
    print(f"  {v}: {stats_rec['coefficients'][i]:10.6f}")

print(f"Adjusted R²: {stats_rec['adj_r_squared']:.6f}")
print(f"AIC: {stats_rec['aic']:.4f}")
print(f"BIC: {stats_rec['bic']:.4f}")

# ---------------------------------------------------
# Full model output
```

```
# ---------------------------------------------------
print("\nFULL MODEL: Y ~ X1 + X2 + X3 + X4 + X5")
print("-" * 80)
print(f"Intercept: {stats_full['intercept']:.6f}")
print("Coefficients:")
for i, v in enumerate(full_vars):
    print(f"  {v}: {stats_full['coefficients'][i]:10.6f}")

print(f"Adjusted R²: {stats_full['adj_r_squared']:.6f}")
print(f"AIC: {stats_full['aic']:.4f}")
print(f"BIC: {stats_full['bic']:.4f}")

# ---------------------------------------------------
# F-test: does X1 actually help?
# ---------------------------------------------------
ssr_reduced = stats_rec["ss_residual"]
ssr_full = stats_full["ss_residual"]
k_full = stats_full["k"]

f_stat = ((ssr_reduced - ssr_full) / 1) / (ssr_full / (n - k_full - 1))
p_val = 1 - scipy_stats.f.cdf(f_stat, 1, n - k_full - 1)

print("\n" + "=" * 80)
print("F-TEST FOR SIGNIFICANCE OF X1")
print("=" * 80)
print(f"F-statistic: {f_stat:.6f}")
print(f"p-value: {p_val:.6f}")
print(
    "Conclusion:",
    "X1 is NOT statistically significant at α = 0.05"
    if p_val > 0.05
    else "X1 is statistically significant at α = 0.05"
)

# ===================================================
# Diagnostic plots
# ===================================================
fig, axes = plt.subplots(2, 3, figsize=(16, 10))

# ---- Residuals vs fitted (recommended) ----
ax = axes[0, 0]
ax.scatter(y_hat_rec, resid_rec, alpha=0.6, edgecolors="k", linewidth=0.5)
ax.axhline(0, linestyle="--", linewidth=1.5)
ax.set_title("Residuals vs Fitted (Recommended)")
ax.set_xlabel("Fitted values")
ax.set_ylabel("Residuals")
ax.grid(True, alpha=0.3)

# ---- Q-Q plot (recommended) ----
ax = axes[0, 1]
scipy_stats.probplot(resid_rec, dist="norm", plot=ax)
ax.set_title("Normal Q-Q Plot (Recommended)")
ax.grid(True, alpha=0.3)

# ---- Scale-location plot (recommended) ----
ax = axes[0, 2]
std_resid = resid_rec / np.std(resid_rec)
ax.scatter(y_hat_rec, np.sqrt(np.abs(std_resid)), alpha=0.6, edgecolors="k", lin
ax.set_title("Scale-Location (Recommended)")
ax.set_xlabel("Fitted values")
```

```
ax.set_ylabel("√|Standardized residuals|")
ax.grid(True, alpha=0.3)

# ---- Residuals vs fitted (full) ----
ax = axes[1, 0]
ax.scatter(y_hat_full, resid_full, alpha=0.6, edgecolors="k", linewidth=0.5)
ax.axhline(0, linestyle="--", linewidth=1.5)
ax.set_title("Residuals vs Fitted (Full)")
ax.set_xlabel("Fitted values")
ax.set_ylabel("Residuals")
ax.grid(True, alpha=0.3)

# ---- Q-Q plot (full) ----
ax = axes[1, 1]
scipy_stats.probplot(resid_full, dist="norm", plot=ax)
ax.set_title("Normal Q-Q Plot (Full)")
ax.grid(True, alpha=0.3)

# ---- Coefficient comparison ----
ax = axes[1, 2]
coef_df = pd.DataFrame({
    "Predictor": full_vars,
    "Full Model": stats_full["coefficients"],
    "Recommended Model": [0] + list(stats_rec["coefficients"])
})

x = np.arange(len(coef_df))
w = 0.35

ax.bar(x - w / 2, coef_df["Full Model"], w, label="Full")
ax.bar(x + w / 2, coef_df["Recommended Model"], w, label="Recommended")

ax.axhline(0, linewidth=0.8)
ax.set_title("Coefficient Comparison")
ax.set_xlabel("Predictor")
ax.set_ylabel("Coefficient value")
ax.set_xticks(x)
ax.set_xticklabels(coef_df["Predictor"])
ax.legend()
ax.grid(True, axis="y", alpha=0.3)

plt.tight_layout()
plt.show()
```

```
================================================================================
FINAL MODEL DIAGNOSTICS AND COMPARISON
================================================================================

RECOMMENDED MODEL: Y ~ X2 + X3 + X4 + X5
--------------------------------------------------------------------------------
Intercept: 1.189339
Coefficients:
  X2:  -0.586083
  X3:   0.559157
  X4:   0.710541
  X5:  -0.196639
Adjusted R²: 0.633974
AIC: -23.1710
BIC: -10.1452

FULL MODEL: Y ~ X1 + X2 + X3 + X4 + X5
--------------------------------------------------------------------------------
Intercept: 1.190212
Coefficients:
  X1:  -0.027785
  X2:  -0.584573
  X3:   0.560272
  X4:   0.710075
  X5:  -0.195747
Adjusted R²: 0.630170
AIC: -21.1952
BIC: -5.5642

================================================================================
F-TEST FOR SIGNIFICANCE OF X1
================================================================================
F-statistic: 0.022709
p-value: 0.880538
Conclusion: X1 is NOT statistically significant at α = 0.05
```



In [ ]: